VerdantAbode CO_2 Control System

System Design

This document describes the project structure and components, general system architecture, system context, component tasks, input parsing, and dispatching actions for the VerdantAbode CO_2 control system.

Project Structure

The project is designed to run on a RP2040 microcontroller, uses the PicoSDK, and FreeRTOS as the OS. Both the PicoSDK and FreeRTOS are managed as submodules.

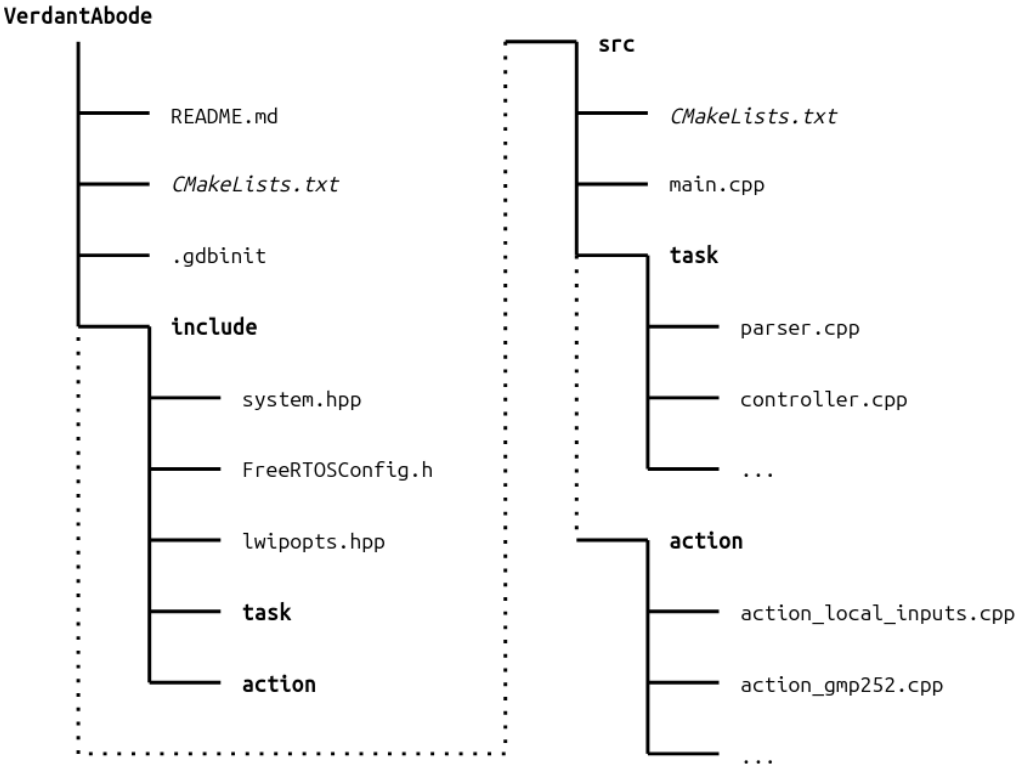


Figure 1: The project directory structure.

FreeRTOS configuration definitions are stored at the root of the *include* directory within *FreeRTOSConfig.h*, and likewise the LwIP definitions are stored in *lwipopts.hpp*. The *system.hpp* file defines the actual system namespace, structures, and variables.

The project code is split into *src* and *include* directories, which store the implementation and definitions respectively. The code is generally further divided into subdirectories where the *task* directory contains all system tasks, and the *actions* directory contains all dispatcher actuated code.

SYSTEM Namespace

The most critical system parameters are easily configurable at the beginning of the *system.hpp* definition file, as can be seen in figure 2.

```
namespace SYSTEM {
    // Variables
    constexpr uint8_t  MAGIC_BYTE = 0x42;   // EEPROM magic
    constexpr uint16_t CO2_TARGET = 800;    // Target CO2 level (ppm)
    constexpr uint16_t CO2_SPAN   = 15;     // +- tolerance around CO2_TARGET

    constexpr uint16_t CO2_FLOOR = 200;     // Lowest allowed CO2 target
    constexpr uint16_t CO2_CEIL  = 1500;    // Highest allowed CO2 target
    constexpr uint16_t CO2_INCR  = 5;       // Delta per ctrl increment
    constexpr uint16_t CO2_CRITICAL = 2000; // Critical CO2 limit

    // Event flags
    constexpr EventBits_t FLAG_CO2_HIGH         = ( 1 << 0 );  // Indicates high CO2
    constexpr EventBits_t FLAG_CO2_LOW          = ( 1 << 1 );  // Indicates low CO2
    constexpr EventBits_t FLAG_WIFI_SETUP       = ( 1 << 2 );  // WIFI setup screen?
    constexpr EventBits_t FLAG_WIFI_PFIELD      = ( 1 << 3 );  // Pass-field selected?
    constexpr EventBits_t FLAG_WIFI_CONNECTED   = ( 1 << 4 );  // Wifi connetion status
    constexpr EventBits_t FLAG_VALVE_OPEN       = ( 1 << 5 );  // CO2 valve is open
    constexpr EventBits_t FLAG_WIFI_SETUP_READY = ( 1 << 6 );  // Wifi ready for connect
```

Figure 2: The *system.hpp* defines fundamental system configuration parameters.

The namespace definition further includes the active system context that is often used as the basis for inter-task communication during operation.

```
struct DATA {
// System State
    EventGroupHandle_t events = xEventGroupCreate();

    uint8_t magic = 0x42;
    int16_t co2_target = SYSTEM::CO2_TARGET;  // Target CO2 level
    char* wifi_ssid[SSID_BUFF_LEN] = {0};     // Wifi SSID
    char* wifi_pass[SSID_BUFF_LEN] = {0};     // Wifi Password
    uint8_t wifi_ssid_pos = 0;
    uint8_t wifi_pass_pos = 0;

    // Sensor & Actuator
    int16_t val_co2  = 0;
    float   val_rh   = 0;
    float   val_temp = 0;
    float   val_pa   = 0;
    int     val_fan  = 0;

    TickType_t time_valve_opened_ms = 0; // When valve was prev. opened
    TickType_t time_valve_closed_ms = 0; // When valve was prev. closed

    // Input Related
    TickType_t prev_input_time_ms = 0;  // When last input was
    char setup_c  = 0;                  // Current Character (setup input fields)
    int ctr_input = 0;                  // Auxiliary ctr for smoothing inputs

    SemaphoreHandle_t mutex_i2c  = xSemaphoreCreateMutex();
    SemaphoreHandle_t mutex_uart = xSemaphoreCreateMutex();

    QueueHandle_t input_queue  = nullptr;  // Local inputs
    QueueHandle_t sdp610_queue = nullptr;  // SDP610::PARAM->que
    QueueHandle_t gmp252_queue = nullptr;  // GMP252::PARAM->que
    QueueHandle_t hmp60_queue  = nullptr;  // HMP60::PARAM->que
    QueueHandle_t mio_queue    = nullptr;  // MIO::PARAM->que

    // Devices
    std::shared_ptr<PicoUart>      uart       = nullptr;
    std::shared_ptr<ModbusClient>  rtu_client = nullptr;

    std::shared_ptr<PicoI2CBus>    i2c_bus = nullptr;
    std::shared_ptr<PicoI2CDevice> i2c_dev = nullptr;
    std::shared_ptr<ssd1306>       display = nullptr;
```

Figure 3: SYSTEM::DATA is the runtime system context structure.

Tasks

The system tasks were all designed to conform to a unified structure, and be as independent as possible; however, some integration work remained unfinished at the end of the project. As such many of the tasks aren't in practice fully independent of the *system.hpp* definitions.

An example of a component task that is as independent as possible is the SDP610 task, which only requires the PicoSDK I²C API functions and FreeRTOS semaphores and queues.

```
QueueHandle_t SDP610::create_task(SemaphoreHandle_t mutex_i2c) {
// Helper to automatically create SDP610 task,
// returns the datapoint queue handle.
    static SDP610::CTX ctx;
    ctx.mutex = mutex_i2c;
    ctx.que = xQueueCreate(SDP610::QUE_LEN, sizeof(SDP610::QUE_ELEMENT));

    // Validation, Registration
    if (ctx.que == nullptr) {
        while (true) printf("[SDP610] Queue is NULL!!!\n");
    }
    vQueueAddToRegistry(ctx.que, "SDP610");

    // Task
    xTaskCreate(SDP610::task, "SDP610", SDP610::STACK_DEPTH,
             (void *) &ctx, SDP610::TASK_PRIORITY, NULL);
    return ctx.que;
}
```

Figure 4: SDP610 task has no project specific dependencies.

Sensor Tasks

The system uses I²C and Modbus (UART) to interface with sensors. Therefore there are two types of sensor tasks, one that uses the UART driver, and one that uses I²C. Of the two Modbus is more particular, because of the need to manage UART TX/RX buffers. If this management is incorrectly done, garbage data will be written to or read from the targeted device register addresses. In practice this is handled by commandeering the bus using the system mutex for UART, then flushing the TX/RX buffers before the read or write operation begins. An example flowchart for a Modbus sensor is shown in the appendix 1 flowchart.

Parser Task

All system inputs are processed by the parser task (*parser.cpp*). Its job is to read available inputs from system context queues and dispatch any obtained data to corresponding actions (found in *src/actions/*). The system context event group flags can provide more specificity as to what action should be taken, like with local inputs.

With the current architecture it's critical that none of the actions are blocking, otherwise the parser stalls. Further improvements could be made with minimal effort to separate actions into their own tasks and use direct task notifications from the parser to signal events. The behaviour of the parser is described in the appendix 2 flowchart.

Controller Task

The main CO_2 controller task is very simple. Its only job is to actuate the CO_2 injection valve or the exhaust fan depending on environmental readings. This is accomplished simply by reading the system context for the latest CO_2 value, and comparing it against the target level. Below is a flowchart demonstrating the logic for its operation.

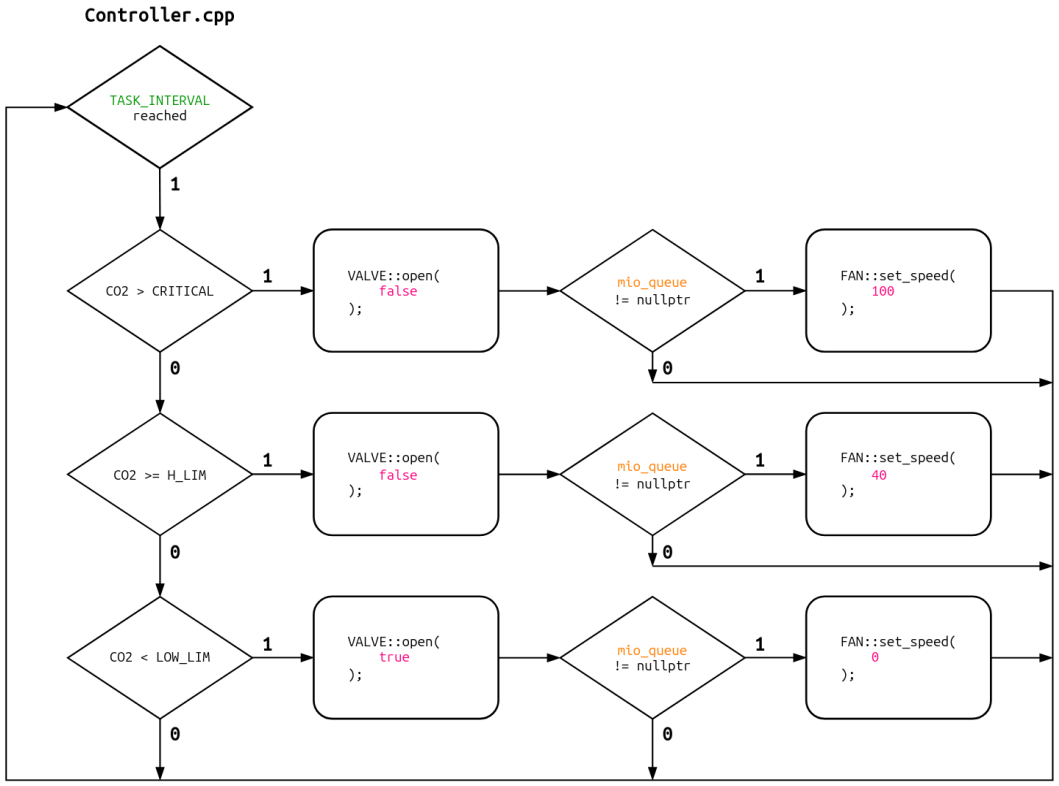


Figure 5: Controller task uses the injection valve and exhaust fan to balance CO_2 levels.

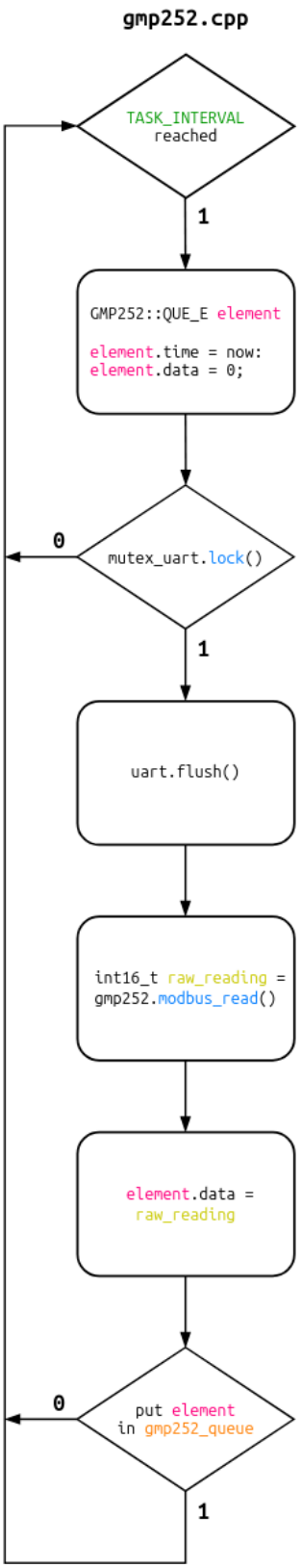
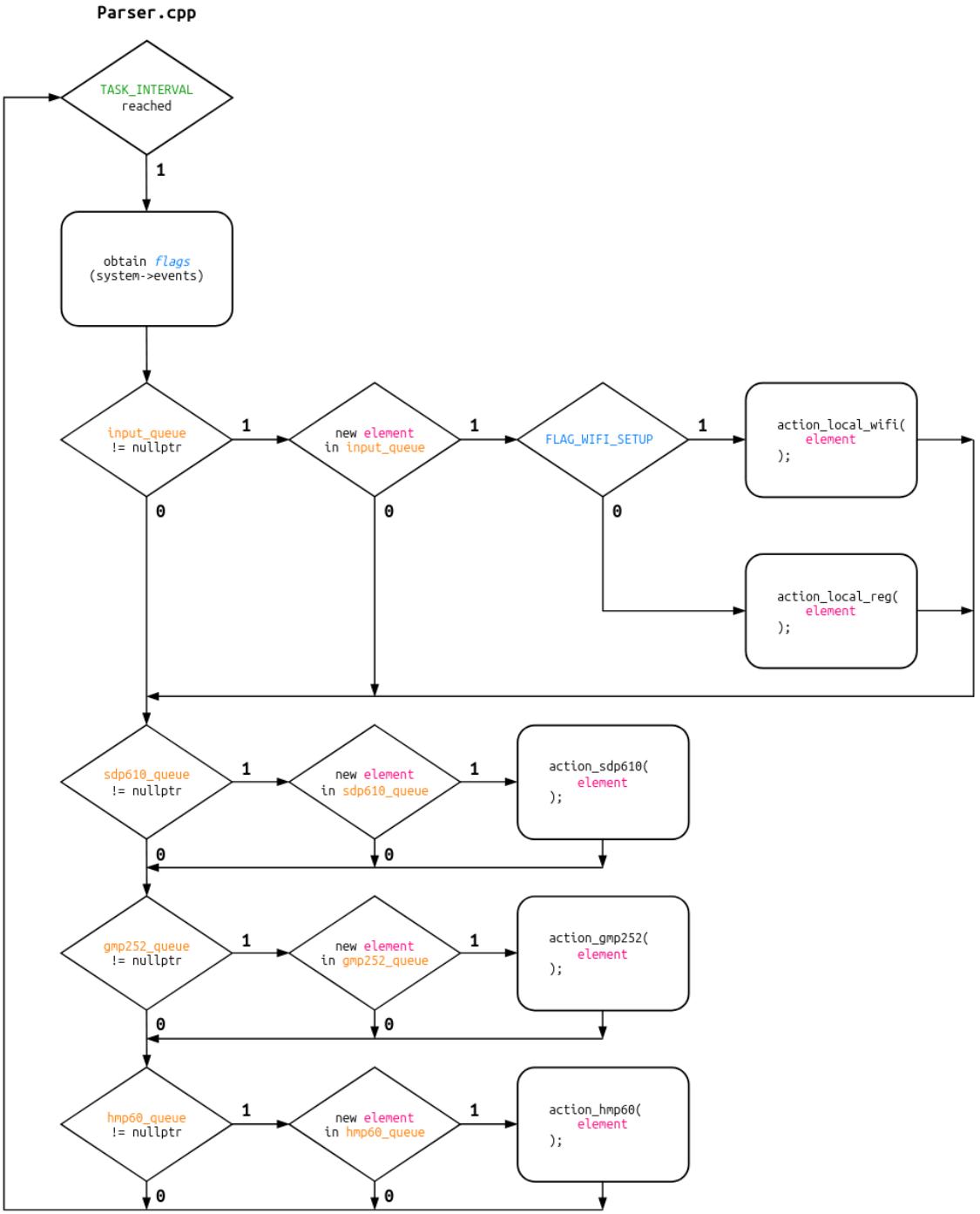
Tuning Priorities and Arrival Rate

The system priorities are not tuned to a satisfactory level, and the queue lengths are not tuned to real world performance of the actual system. The simulator is not a good benchmark for those details, so the currently used settings are larger than what is probably needed in practice.

Network Connectivity and ThingSpeak

The network connectivity is functional, and it's possible to configure it using the local inputs and the screen. Sending data to the ThingSpeak is also tested to be functional, but talkback queue is not implemented for sending remote commands to the system. TLS remains unimplemented, so the system shouldn't be used to send data to a backend, as that would expose the API-key in production environments.

Appendix 1: GMP252, Modbus (Flowchart)



Appendix 2: Parser Task (Flowchart)